Задача 1. Параметры командной строки

Источник:	базовая
Имя входного файла:	argv
Имя выходного файла:	stdout/stderr
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В этой задаче нужно выполнить одну арифметическую операцию: сложение, вычитание или умножение. В некоторых случаях нужно делать эту операцию по модулю, в некоторых — нет.

Входные данные передаются через параметры командной строки. Операция определяется типом и двумя аргументами, которые идут в командной строке подряд в точности в этом порядке.

Тип всегда является одним из:

- **add** — нужно сложить два числа,
- **sub** — нужно вычесть из первого числа второе,
- **mul** — нужно найти произведение двух чисел.

Например, если исполняемый файл вашего решения называется sol.exe, то следующая команда предписывает вычесть из числа 173 число 546 (результат равен -373):

sol.exe sub 173 546

Кроме того, в командной строке может быть задан модуль, по которому нужно выполнять операцию. Модуль задаётся параметром -m и числом, которое задаёт модуль — они также всегда идут подряд. Модуль может быть задан либо в самом начале командной строки, либо в самом конце. Например:

sol.exe sub 135 1793846 -m 100

sol.exe -m 100 sub 135 1793846

Гарантируется, что других параметров в командной строке нет. Все числа целые, не превышают 109 по абсолютной величине. Число, задающее модуль, также положительно и больше единицы.

Одно целое число, результат арифметической операции, нужно вывести в стандартный поток вывода stdout. Если операция выполняется по модулю, то это число должен быть неотрицательным и меньше модуля. Кроме того, решение должно завершиться с нулевым кодом возврата.

Пользователи часто запускают программы без параметров, и от вас требуется также поддержать этот случай. Если решение запускается без параметров, то оно должно:

- ничего не выводить в stdout,
- вывести сообщение No parameters specified. в стандартный поток ошибок stderr,
- вернуть код возврата 13, сообщая таким образом о неверном использовании.

Примеры

argv	stdout/stderr
sol.exe mul 1000000000 17345	17345000000000
sol.exe	No parameters specified.
sol.exe sub 135 1793846 -m 100	89

Задача 2. Система линейных уравнений

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дана система из N линейных уравнений и N переменных:

$$\begin{gathered} a_{1,1}x_1+a_{1,2}x_2+\cdots+a_{1,N}x_N=b_1 \\ a_{2,1}x_1+a_{2,2}x_2+\cdots+a_{2,N}x_N=b_2 \\ \vdots \\ a_{N,1}x_1+a_{N,2}x_2+\cdots+a_{N,N}x_N=b_N \end{gathered}$$

Требуется решить данную систему *двумя методами*: ***методом Гаусса*** и ***итерационным методом***. Гарантируется, что данная система имеет единственное решение. Объяснить, когда какой из методов стоит использовать и почему.

Формат входных данных

В первой строке задано число N — количество уравнений и переменных ($1 \le N \le 200$).
Следующие N строк содержат описывают систему уравнений. i-ая строка содержит по (N+1) вещественному числу: коэффициенты $a_{i,1}, a_{i,2}x_2, \ldots, a_{i,N}$ и значение правой части системы b_i.

Формат выходных данных

Выведите N строк: значения переменных $x_1, x_2, \cdots, x_N$.
Ответ будет считаться правильным, если абсолютная погрешность в каждой строке не будет превышать 10^{-3}:

Примеры

input.txt	output.txt
3 2 1 -1 8 -3 -1 2 -11 -2 1 2 -3	2.0000000000 3.0000000000 -1.0000000000
2 0.2 0.3 0.4 0.5 0.6 0.8	-0.0000000000 1.3333333333

Задача 3. Параметры командной строки 2

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	разумное
Ограничение по памяти:	разумное

В этой задаче предлагается реализовать алгоритм выделения параметров из командной строки в простом случае. Командная строка записана в единственной строке входного файла input.txt. Ваша программа должна прочитать её, и вывести в выходной файл отдельные параметры.
Командная строка будет содержать только следующие символы:

- латинские буквы (большие и маленькие) и цифры,
- пробел (ASCII 32),
- двойные кавычки (ASCII 34).

Параметры записываются в командную строку подряд в порядке следования. Между каждыми двумя соседними параметрами вставляется один или несколько пробелов, чтобы отделить их друг от друга.
Параметр может быть целиком заключён в двойные кавычки. Если внутри параметра есть пробелы, он заключён в двойные кавычки обязательно. Символов двойных кавычек в самих параметрах нет — ими лишь можно окружать параметры.
Все параметры непустые, хотя количество параметров может быть равно нулю. Также произвольное количество пробелов может быть добавлено в начало и в конец командной строки.
Командная строка, которую необходимо разобрать, записана в единственной строке входного файла. Её длина не превышает 100 символов. Учтите, что в этой командной строке отсутствует имя исполняемого файла. В выходной файл нужно вывести искомые параметры командной строки, по одному параметру в строке. Каждый параметр должен быть заключён в квадратные скобки. Особый нулевой параметр (саму командную строку) выводить не нужно.
Данная задача проверяется на четырёх разных компиляторах:
1. cl — компилятор C из Microsoft Visual Studio (основной компилятор в системе Windows).
2. gcc — компилятор C из GNU Compiler Collection (основной компилятор в системе Linux), сборка Mingw-w64.
3. clang — компилятор C из Clang для LLVM (основной компилятор в системах MacOSX и FreeBSD), команда clang.exe из официальной виндовой сборки.
4. tcc — малоизвестный и очень забавный компилятор Tiny C Compiler.
Решение должно компилироваться, верно работать на всех четырёх компиляторах, ***Семинаристам*** должен быть продемонстрирован компиляция и запуск с помощью всех четырёх компиляторах и рассказана теория об этих компиляторах.

Примеры

input.txt	output.txt
"abc d" e	[abc d] [e]
"long p " "p 2" "sht" s pars	[long p] [p 2] [sht] [s] [pars]

Задача 4. Система сравнений по модулю

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дана система сравнений по модулю простого числа P.

Если у системы существует единственное решение, нужно его вывести. В противном случае нужно вывести количество различных решений. Поскольку их может быть много, нужно найти количество по модулю 1 000 000 007.

Формат входных данных

В первой строке задано три целых числа: n — количество переменных, k — количество уравнений и P — модуль ($1 \le n, k \le 300$, $2 \le P < 10^9$). Гарантируется, что число P — простое. Следующие k строк описывают уравнения системы. В каждой строке (n+1) целых чисел в диапазоне от 0 до ($P - 1$) включительно. Если обозначить i-ое из этих чисел через A_i, то уравнение выглядит так:

$$\left(A_1 x_1 + A_2 x_2 + \cdots + A_n x_n\right) mod\, P = A_{n+1}$$

В уравнениях неизвестными переменными считаются $x_1, x_2, \cdots, x_n$. В решении значения всех переменных должны быть в диапазоне от 0 до ($P - 1$) включительно.

Формат выходных данных

Если решение существует и единственно, то нужно его вывести. В этом случае выведите n целых чисел в диапазоне от 0 до ($P - 1$) — значения переменных $x_1, x_2, \cdots, x_n$.
Если решения нет, выведите одно целое число ноль. Если решений много, выведите остаток от деления количества решений на ($10^9 + 7$).

Примеры

input.txt	output.txt
4 4 7 1 2 3 4 5 0 1 4 0 1 0 0 1 1 2 0 0 0 1 6	1 1 3 3 6
3 2 2 1 1 0 1 1 1 0 1	4

Комментарий

Первый пример совпадает с первым тестом, а второй пример не совпадает со вторым.

Задача 5. Поиск выпуклой оболочки

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Требуется найти выпуклую оболочку множества точек на плоскости алгоритмом Джарвиса

Формат входных данных

Первая строка — целое число n ($3 \le n \le 100\,000$) — количество точек.
Следующие n строк содержат по два целых числа x_i y_i — координаты точек ($-10^9 \le x_i, y_i \le 10^9$).
Гарантируется, что все точки различны. Может быть до 3 коллинеарных точек подряд.

Формат выходных данных

В первой строке выведите количество точек k на выпуклой оболочке.

Далее выведите k строк — координаты вершин выпуклой оболочки в порядке обхода против часовой стрелки, начиная с самой левой точки (при нескольких — самой нижней из них).

Пример

input.txt	output.txt
8 0 0 10 0 5 10 2 2 3 3 4 4 5 5 6 3	3 0 0 10 0 5 10

Задача 6. Приближение полиномом

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Вам нужно сделать подбор кривой для заданного множества экспериментальных точек. Предположим, что дано множество из m точек

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n),$$

где значения y, содержат ошибки измерений. Нужно найти функцию F(x), такую, что ошибки аппроксимации

$$\eta_i = F(x_i) - y_i$$

малы при i = 1, 2, …, m. Вид функции F зависит от рассматриваемой задачи, и здесь мы будем считать, что она имеет вид линейной взвешенной суммы

$$F(x) = \sum_{j=1}^{n} c_j f_j(x),$$

где количество слагаемых n и набор базисных функций (basis functions) f выбираются на основе знаний о рассматриваемой задаче. Зачастую в качестве базисных функций выбираются $f_j(x) = x^{j-1}$, т.е. функция F представляет собой полином степени n - 1 от x:

$$F(x) = c_1 + c_2 x + c_3 x^2 + \cdots + c_n x^{n-1}.$$

Таким образом, для заданных n экспериментальных точек $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ необходимо вычислить n коэффициентов c_1, c_2,…, c_n, минимизирующих ошибки приближения $\eta_1, \eta_2, \ldots, \eta_n$. ***Используйте теорему Шура!***

Формат входного файла

Первая строка содержит число n – количество точек (1 ≤ n ≤ 100). Следующие n строк содержат пары чисел (x_i, y_i).

Формат выходного файла

Через пробел вывести c_1, c_2,…, c_n с точностью не хуже 10^{-3}.

Пример

input.txt	output.txt
3 1 3 2 5 3 7	1.0000000000 2.0000000000 0.0000000000

Задача 7. Интерполяция

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Одно из применений полиномиальной интерполяции — это замена трудновычислимой функции простым многочленом. Функция может вычисляться очень медленно, или вообще требовать несохраняемых данных для своего вычисления. Так, например, при пересечении двух криволинейных поверхностей в геометрическом моделировании кривую пересечения не сохраняют полностью, а сохраняют только результат интерполяции.

В этой задаче в качестве трудновычислимой функции будем использовать функцию:

$$f(x)=\sum_{i=1}^{N} w_i \cos(a_i x+b)$$

Параметры подобраны так, что эта функция гладкая и её значение и производные невелики. Требуется вычислить функцию $f(x)$ в M заданных точках, используя заранее построенные интерполяционные многочлены. Необходимо разбить область определения на много отрезков, и на каждом отрезке посчитать свой интерполяционный многочлен невысокой степени. Точный подбор параметров (количество отрезков и степень) — это ваша задача.

Формат входного файла

В первой строке дано два целых числа: N — количество слагаемых в функции $f(x)$, M — количество точек, в которых нужно узнать значение функции ($1 \le N \le 10\,000$, $1 \le M \le 100\,000$).

В следующих N строках даны параметры слагаемых в функции. В каждой строке записано три вещественных числа w_i, a_i и b_i. При этом $|w_i| \le 1$, $|a_i|, |b_i| \le 20$.

В оставшихся M строках даны значения x, для которых нужно вычислить функцию $f(x)$.

В каждой строке записано одно вещественное число в диапазоне [0; 1].

Формат выходного файла

Нужно вывести ровно M вещественных чисел, по одному числу в строке. Рекомендуется выводить числа с максимальной точностью.

Абсолютная ошибка каждого вычисленного значения не должна превышать 10^{-12}.

Пример

input.txt	output.txt
2 5 0.76 1.0 0.5 -0.34 2.0 0.0 0.0 0.5 1.0 0.7853981634 0.5707963268	0.32696274703668332107 0.22692696846461868354 0.19525019769348261889 0.21397004366832683697 0.22287348491291761321

Задача 8. Параметры командной строки 3

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В программировании очень важно поддерживать совместимость между программами. Бывает необходимо поддержать поведение другой программы идеально точно, даже если это поведение кажется странным.

В этой задаче предлагается реализовать алгоритм выделения параметров из командной строки в современной программе, собранной с помощью Microsoft Visual C. Командная строка записана в единственной строке входного файла input.txt. Ваша программа должна прочитать её, и вывести в выходной файл отдельные параметры.

Для простоты, командная строка будет содержать только следующие символы:

- латинские буквы (большие и маленькие) и цифры,
- двоеточие (ASCII 58) и точка (ASCII 46),
- пробел (ASCII 32),
- двойные кавычки (ASCII 34),
- обратный слэш (ASCII 92).

Правила разбора командной строки программой описаны в документации Microsoft. Следует заметить, что в реальном мире документация порой оказывается неполной или неверной. Скорее всего, вам понадобится дополнительная статья для полного понимания поведения.

Учтите, что вы можете очень легко самостоятельно проверить, какой ответ является правильным для любой заданной строки. Для этого создайте простую программу test.c, которая распечатывает содержимое массива argv, исключая нулевой элемент. Скомпилируйте программу с помощью Visual C, затем напишите в командной строке text.exe, пробел, строку из входных данных, и нажмите Enter.

ВАЖНО: программа должна быть собрана с помощью Visual C, т.к. в других компиляторах правила выделения параметров немного другие! Он пишет параметры в файл output.txt, и должен работать не только в Windows, но даже в Linux + Wine.

Командная строка, которую необходимо разобрать, записана в единственной строке входного файла. Её длина не превышает 100 символов. Учтите, что в этой командной строке отсутствует имя исполняемого файла. В выходной файл нужно вывести искомые параметры командной строки, по одному параметру в строке. Каждый параметр должен быть заключён в квадратные скобки. Особый нулевой параметр (саму командную строку) выводить не нужно.

Данная задача проверяется ***с семинаристами*** на четырёх разных компиляторах. Решение должно компилироваться и верно работать на всех четырёх компиляторах**.**

Примеры

input.txt	output.txt
"abc d" e	[abc] [d] [e]
a\\b d"e f"g h	[a\\b] [de fg] [h]
a\\\"b c d	[a\"b] [c] [d]

Задача 9. Система сравнений по модулю +

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дана система сравнений по модулю **произвольного** числа M.

Если у системы существует единственное решение, нужно его вывести. В противном случае нужно вывести количество различных решений. Поскольку их может быть много, нужно найти количество по модулю 1 000 000 007.

Формат входных данных

В первой строке задано три целых числа: n — количество переменных, k — количество уравнений и P — модуль ($1 \le n, k \le 200$, $2 \le M \le 10^9$). M может **не** быть простым числом!
Следующие k строк описывают уравнения системы. В каждой строке (n+1) целых чисел в диапазоне от 0 до (M − 1) включительно. Если обозначить i-ое из этих чисел через A_i, то уравнение выглядит так:

$$\left(A_1 x_1 + A_2 x_2 + \cdots + A_n x_n\right) mod\, M = A_{n+1}$$

В уравнениях неизвестными переменными считаются $x_1, x_2, \cdots, x_n$. В решении значения всех переменных должны быть в диапазоне от 0 до (M − 1) включительно.

Формат выходных данных

В первой строке выведите остаток от деления количества решений на (10^9 + 7). Кроме того, требуется вывести решение, если хотя бы одно существует. В этом случае выведите n целых чисел в диапазоне от 0 до (M − 1) — значения переменных $x_1, x_2, \cdots, x_n$.
Если решений много, можно вывести любое из них.

Примеры

input.txt	output.txt
4 4 7 1 2 3 4 5 0 1 4 0 1 0 0 1 1 2 0 0 0 1 6	1 1 3 3 6
3 2 2 1 1 0 1 1 1 0 1	4 1 0 0

Комментарий

Первый пример совпадает с первым тестом, а второй пример **не** совпадает со вторым.